

DSC140B_HW3_Q6_TODO

June 5, 2026

1 Setup

```
[1]: import torch
import torch.nn as nn
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms
from torchvision import datasets
import matplotlib.pyplot as plt
import numpy as np
from torch.utils.data import DataLoader
```

```
[2]: # Set random seed for reproducibility
torch.manual_seed(42)
```

```
[2]: <torch._C.Generator at 0x215e4ed7090>
```

2 Hyperparameters

```
[3]: # DO NOT CHANGE
batch_size = 128
learning_rate = 1e-3
num_epochs = 10
input_size = 28 * 28 # FashionMNIST images are 28x28 pixels
hidden_sizes = [512, 256, 128] # Three hidden layers
num_classes = 10 # 10 classes in FashionMNIST
```

3 Data

```
[4]: # Define transforms for the training and test sets
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))
])
```

```
[5]: # Complete this with appropriate batch_size
trainset = datasets.FashionMNIST(
    root = "data",
    train = True,
    download = True,
    transform = transform
)
trainloader = DataLoader(trainset, batch_size=batch_size, shuffle=True)

testset = datasets.FashionMNIST(
    root = "data",
    train = False,
    download = True,
    transform = transform
)
testloader = DataLoader(testset, batch_size=batch_size, shuffle=False)
```

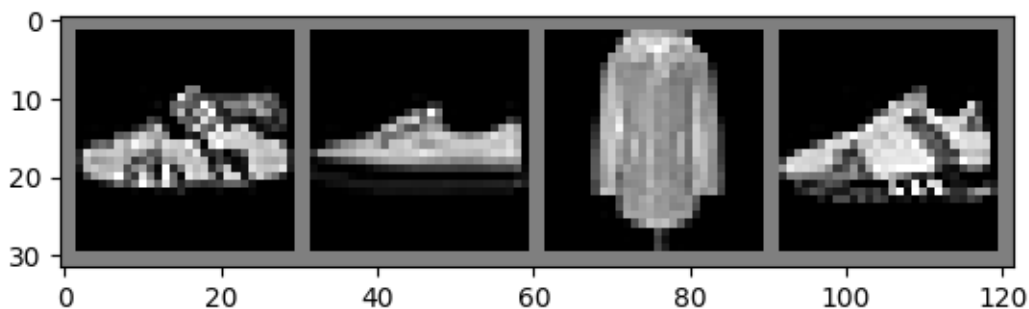
```
[6]: # Class labels
classes = ('T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat',
          'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle Boot')
```

```
[7]: # Function to show an image
def imshow(img):
    img = img / 2 + 0.5 # unnormalize
    npimg = img.numpy()
    plt.imshow(np.transpose(npimg, (1, 2, 0)))
    plt.show()
```

```
[8]: # Get random training images
dataiter = iter(trainloader)
images, labels = next(dataiter)

# Show images
imshow(torchvision.utils.make_grid(images[:4]))

# Print labels
print(', '.join(f'{classes[labels[j]]:5s}' for j in range(4)))
```



Sandal, Sneaker, Coat , Sneaker

4 MLP

```
[9]: class MLP(nn.Module):
    def __init__(self, input_size, hidden_sizes, num_classes):
        super(MLP, self).__init__()
        # The architecture should have:
        # - An input layer that connects to the first hidden layer (512 neurons)
        # - ReLU activation after the first hidden layer
        # - A second hidden layer (256 neurons)
        # - ReLU activation after the second hidden layer
        # - A third hidden layer (128 neurons)
        # - ReLU activation after the third hidden layer
        # - An output layer with 10 neurons (one for each class)
        # Hint: Use nn.Sequential, nn.Linear, nn.ReLU for a clean implementation

        layers = []
        layers.append(nn.Linear(input_size, hidden_sizes[0]))
        layers.append(nn.ReLU())
        for i in range(len(hidden_sizes)-1):
            layers.append(nn.Linear(hidden_sizes[i], hidden_sizes[i+1]))
            layers.append(nn.ReLU())
        layers.append(nn.Linear(hidden_sizes[-1], num_classes))

        self.layers = nn.Sequential(*layers)

    def forward(self, x):
        x = x.view(x.size(0), -1)
        x = self.layers(x)

        return x
```

5 CNN

```
[10]: class CNN(nn.Module):
    """
    A simple Convolutional Neural Network (CNN) for classifying FashionMNIST_
    ↪ images.

    Architecture:
    - Two convolutional layers with ReLU activation and max pooling.
    - A fully connected classification head.
    """
```

```

def __init__(self, num_classes):
    super(CNN, self).__init__()

    self.conv1 = nn.
↪Conv2d(in_channels=1,out_channels=64,kernel_size=3,stride=1,padding=1)
    self.relu1 = nn.ReLU()
    self.maxpool1 = nn.MaxPool2d(kernel_size=2)

    self.conv2 = nn.
↪Conv2d(in_channels=64,out_channels=64,kernel_size=3,stride=1,padding=1)
    self.relu2 = nn.ReLU()
    self.maxpool2 = nn.MaxPool2d(kernel_size=2)

    self.flatten = nn.Flatten()

    # Hint: Input dimension of the linear layer can be calculated
    # from previous operations
    self.fc = nn.Linear(64 * 7 * 7, num_classes)

def forward(self, x):
    """
    Forward pass of the CNN.

    Args:
        x (torch.Tensor): Input batch of shape (batch_size, 1, 28, 28)

    Returns:
        torch.Tensor: Logits of shape (batch_size, num_classes)
    """
    x = self.conv1(x)
    x = self.relu1(x)
    x = self.maxpool1(x)

    x = self.conv2(x)
    x = self.relu2(x)
    x = self.maxpool2(x)

    x = self.flatten(x)
    x = self.fc(x)

    return x

```

6 Train

```
[11]: # Initialize the model
model_mlp = MLP(input_size, hidden_sizes, num_classes)
model_cnn = CNN(num_classes)

[12]: # Check if GPU is available
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {device}")

# Move models to GPU, if available
model_mlp = model_mlp.to(device)
model_cnn = model_cnn.to(device)
```

Using device: cuda

```
[13]: # Implement the training loop
def train(model, trainloader, num_epochs):

    # Define loss function and optimizer (Adam)
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(model.parameters(), lr=learning_rate)

    # Set model to training mode
    model.train()

    # Lists to track progress
    train_losses = []
    train_accs = []

    for epoch in range(num_epochs):
        running_loss = 0.0
        correct = 0
        total = 0
        epoch_loss_total = 0.0
        epoch_correct = 0
        epoch_total = 0

        for i, data in enumerate(trainloader, 0):
            # Get the inputs and move them to the device
            inputs, labels = data
            inputs, labels = inputs.to(device), labels.to(device)

            # Zero the parameter gradients
            optimizer.zero_grad()

            # Forward pass and compute loss
            outputs = model(inputs)
```

```

        loss = criterion(outputs, labels)

        # Backward pass and optimizer step
        loss.backward()
        optimizer.step()

        # Track statistics
        running_loss += loss.item()
        _, predicted = torch.max(outputs.data, 1)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()
        epoch_loss_total += loss.item()
        epoch_total += labels.size(0)
        epoch_correct += (predicted == labels).sum().item()

        # Print statistics every 100 mini-batches
        if i % 100 == 99:
            batch_loss = running_loss / 100
            batch_acc = 100 * correct / total
            print(f'Epoch: {epoch + 1}, Batch: {i + 1}, Loss: {batch_loss:.
↪3f}, Accuracy: {batch_acc:.2f}%')
            running_loss = 0.0
            correct = 0
            total = 0

        # Calculate epoch-level statistics
        epoch_loss = epoch_loss_total / len(trainloader)
        epoch_acc = 100 * epoch_correct / epoch_total
        train_losses.append(epoch_loss)
        train_accs.append(epoch_acc)

        print(f'Epoch {epoch + 1} completed. Train Loss: {epoch_loss:.3f},
↪Train Accuracy: {epoch_acc:.2f}%')

    print('Finished Training')
    return train_losses, train_accs

```

```

[14]: # Implement the evaluation function
def evaluate(model, dataloader):

```

```

    # Define loss function
    criterion = nn.CrossEntropyLoss()

    model.eval()
    running_loss = 0.0
    correct = 0
    total = 0

```

```

with torch.no_grad():
    for data in dataloader:
        # Get the inputs and move them to the device
        images, labels = data
        images, labels = images.to(device), labels.to(device)

        # Forward pass and compute loss
        outputs = model(images)
        loss = criterion(outputs, labels)

        # Track statistics
        running_loss += loss.item()
        _, predicted = torch.max(outputs.data, 1)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()

    avg_loss = running_loss / len(dataloader)
    accuracy = 100 * correct / total

    return avg_loss, accuracy

```

```

[15]: print("Training MLP...")
      train_losses_mlp, train_accs_mlp = train(model_mlp, trainloader, num_epochs)

      print("Training CNN...")
      train_losses_cnn, train_accs_cnn = train(model_cnn, trainloader, num_epochs)

```

```

Training MLP...
Epoch: 1, Batch: 100, Loss: 0.767, Accuracy: 71.90%
Epoch: 1, Batch: 200, Loss: 0.509, Accuracy: 81.06%
Epoch: 1, Batch: 300, Loss: 0.454, Accuracy: 83.71%
Epoch: 1, Batch: 400, Loss: 0.428, Accuracy: 84.38%
Epoch 1 completed. Train Loss: 0.520, Train Accuracy: 81.02%
Epoch: 2, Batch: 100, Loss: 0.383, Accuracy: 86.02%
Epoch: 2, Batch: 200, Loss: 0.372, Accuracy: 86.38%
Epoch: 2, Batch: 300, Loss: 0.379, Accuracy: 86.16%
Epoch: 2, Batch: 400, Loss: 0.366, Accuracy: 86.51%
Epoch 2 completed. Train Loss: 0.372, Train Accuracy: 86.35%
Epoch: 3, Batch: 100, Loss: 0.336, Accuracy: 87.66%
Epoch: 3, Batch: 200, Loss: 0.333, Accuracy: 87.62%
Epoch: 3, Batch: 300, Loss: 0.337, Accuracy: 87.59%
Epoch: 3, Batch: 400, Loss: 0.347, Accuracy: 87.17%
Epoch 3 completed. Train Loss: 0.336, Train Accuracy: 87.56%
Epoch: 4, Batch: 100, Loss: 0.308, Accuracy: 88.52%
Epoch: 4, Batch: 200, Loss: 0.310, Accuracy: 88.57%
Epoch: 4, Batch: 300, Loss: 0.321, Accuracy: 88.28%
Epoch: 4, Batch: 400, Loss: 0.302, Accuracy: 88.88%

```

Epoch 4 completed. Train Loss: 0.310, Train Accuracy: 88.58%
 Epoch: 5, Batch: 100, Loss: 0.281, Accuracy: 89.47%
 Epoch: 5, Batch: 200, Loss: 0.297, Accuracy: 88.96%
 Epoch: 5, Batch: 300, Loss: 0.296, Accuracy: 88.83%
 Epoch: 5, Batch: 400, Loss: 0.287, Accuracy: 89.34%
 Epoch 5 completed. Train Loss: 0.289, Train Accuracy: 89.27%
 Epoch: 6, Batch: 100, Loss: 0.272, Accuracy: 90.05%
 Epoch: 6, Batch: 200, Loss: 0.267, Accuracy: 90.02%
 Epoch: 6, Batch: 300, Loss: 0.262, Accuracy: 90.08%
 Epoch: 6, Batch: 400, Loss: 0.275, Accuracy: 89.77%
 Epoch 6 completed. Train Loss: 0.270, Train Accuracy: 89.95%
 Epoch: 7, Batch: 100, Loss: 0.252, Accuracy: 90.59%
 Epoch: 7, Batch: 200, Loss: 0.257, Accuracy: 90.41%
 Epoch: 7, Batch: 300, Loss: 0.270, Accuracy: 89.93%
 Epoch: 7, Batch: 400, Loss: 0.241, Accuracy: 90.91%
 Epoch 7 completed. Train Loss: 0.257, Train Accuracy: 90.42%
 Epoch: 8, Batch: 100, Loss: 0.245, Accuracy: 90.79%
 Epoch: 8, Batch: 200, Loss: 0.241, Accuracy: 90.77%
 Epoch: 8, Batch: 300, Loss: 0.242, Accuracy: 91.04%
 Epoch: 8, Batch: 400, Loss: 0.249, Accuracy: 90.73%
 Epoch 8 completed. Train Loss: 0.244, Train Accuracy: 90.87%
 Epoch: 9, Batch: 100, Loss: 0.224, Accuracy: 91.53%
 Epoch: 9, Batch: 200, Loss: 0.222, Accuracy: 91.55%
 Epoch: 9, Batch: 300, Loss: 0.231, Accuracy: 91.38%
 Epoch: 9, Batch: 400, Loss: 0.237, Accuracy: 91.10%
 Epoch 9 completed. Train Loss: 0.229, Train Accuracy: 91.29%
 Epoch: 10, Batch: 100, Loss: 0.209, Accuracy: 91.86%
 Epoch: 10, Batch: 200, Loss: 0.217, Accuracy: 91.74%
 Epoch: 10, Batch: 300, Loss: 0.216, Accuracy: 91.87%
 Epoch: 10, Batch: 400, Loss: 0.220, Accuracy: 91.61%
 Epoch 10 completed. Train Loss: 0.217, Train Accuracy: 91.77%
 Finished Training
 Training CNN...
 Epoch: 1, Batch: 100, Loss: 0.719, Accuracy: 74.76%
 Epoch: 1, Batch: 200, Loss: 0.432, Accuracy: 84.52%
 Epoch: 1, Batch: 300, Loss: 0.393, Accuracy: 86.02%
 Epoch: 1, Batch: 400, Loss: 0.359, Accuracy: 87.22%
 Epoch 1 completed. Train Loss: 0.457, Train Accuracy: 83.79%
 Epoch: 2, Batch: 100, Loss: 0.315, Accuracy: 88.83%
 Epoch: 2, Batch: 200, Loss: 0.309, Accuracy: 89.11%
 Epoch: 2, Batch: 300, Loss: 0.296, Accuracy: 89.23%
 Epoch: 2, Batch: 400, Loss: 0.288, Accuracy: 89.99%
 Epoch 2 completed. Train Loss: 0.301, Train Accuracy: 89.29%
 Epoch: 3, Batch: 100, Loss: 0.268, Accuracy: 90.46%
 Epoch: 3, Batch: 200, Loss: 0.273, Accuracy: 90.58%
 Epoch: 3, Batch: 300, Loss: 0.246, Accuracy: 91.29%
 Epoch: 3, Batch: 400, Loss: 0.256, Accuracy: 90.97%
 Epoch 3 completed. Train Loss: 0.261, Train Accuracy: 90.80%

Epoch: 4, Batch: 100, Loss: 0.238, Accuracy: 91.98%
 Epoch: 4, Batch: 200, Loss: 0.235, Accuracy: 91.50%
 Epoch: 4, Batch: 300, Loss: 0.223, Accuracy: 91.83%
 Epoch: 4, Batch: 400, Loss: 0.223, Accuracy: 91.98%
 Epoch 4 completed. Train Loss: 0.231, Train Accuracy: 91.79%
 Epoch: 5, Batch: 100, Loss: 0.201, Accuracy: 92.70%
 Epoch: 5, Batch: 200, Loss: 0.206, Accuracy: 92.55%
 Epoch: 5, Batch: 300, Loss: 0.216, Accuracy: 92.15%
 Epoch: 5, Batch: 400, Loss: 0.213, Accuracy: 92.55%
 Epoch 5 completed. Train Loss: 0.209, Train Accuracy: 92.55%
 Epoch: 6, Batch: 100, Loss: 0.184, Accuracy: 93.49%
 Epoch: 6, Batch: 200, Loss: 0.197, Accuracy: 92.97%
 Epoch: 6, Batch: 300, Loss: 0.197, Accuracy: 93.04%
 Epoch: 6, Batch: 400, Loss: 0.197, Accuracy: 92.99%
 Epoch 6 completed. Train Loss: 0.193, Train Accuracy: 93.16%
 Epoch: 7, Batch: 100, Loss: 0.176, Accuracy: 93.78%
 Epoch: 7, Batch: 200, Loss: 0.173, Accuracy: 93.78%
 Epoch: 7, Batch: 300, Loss: 0.183, Accuracy: 93.67%
 Epoch: 7, Batch: 400, Loss: 0.175, Accuracy: 93.70%
 Epoch 7 completed. Train Loss: 0.176, Train Accuracy: 93.74%
 Epoch: 8, Batch: 100, Loss: 0.159, Accuracy: 94.24%
 Epoch: 8, Batch: 200, Loss: 0.170, Accuracy: 93.88%
 Epoch: 8, Batch: 300, Loss: 0.159, Accuracy: 94.36%
 Epoch: 8, Batch: 400, Loss: 0.169, Accuracy: 93.91%
 Epoch 8 completed. Train Loss: 0.165, Train Accuracy: 94.01%
 Epoch: 9, Batch: 100, Loss: 0.147, Accuracy: 94.93%
 Epoch: 9, Batch: 200, Loss: 0.154, Accuracy: 94.41%
 Epoch: 9, Batch: 300, Loss: 0.157, Accuracy: 94.20%
 Epoch: 9, Batch: 400, Loss: 0.151, Accuracy: 94.73%
 Epoch 9 completed. Train Loss: 0.152, Train Accuracy: 94.56%
 Epoch: 10, Batch: 100, Loss: 0.140, Accuracy: 95.13%
 Epoch: 10, Batch: 200, Loss: 0.141, Accuracy: 95.02%
 Epoch: 10, Batch: 300, Loss: 0.146, Accuracy: 94.87%
 Epoch: 10, Batch: 400, Loss: 0.148, Accuracy: 94.70%
 Epoch 10 completed. Train Loss: 0.143, Train Accuracy: 94.93%
 Finished Training

7 Evaluate

```

[16]: # Test the model
print("Evaluating MLP...")
test_loss_mlp, test_acc_mlp = evaluate(model_mlp, testloader)
print(f'MLP: Test Loss: {test_loss_mlp:.3f}, Test Accuracy: {test_acc_mlp:.2f}%')

print("Evaluating CNN...")
test_loss_cnn, test_acc_cnn = evaluate(model_cnn, testloader)
  
```

```
print(f'CNN: Test Loss: {test_loss_cnn:.3f}, Test Accuracy: {test_acc_cnn:.  
↪2f}%')
```

Evaluating MLP...

MLP: Test Loss: 0.323, Test Accuracy: 88.63%

Evaluating CNN...

CNN: Test Loss: 0.238, Test Accuracy: 91.76%

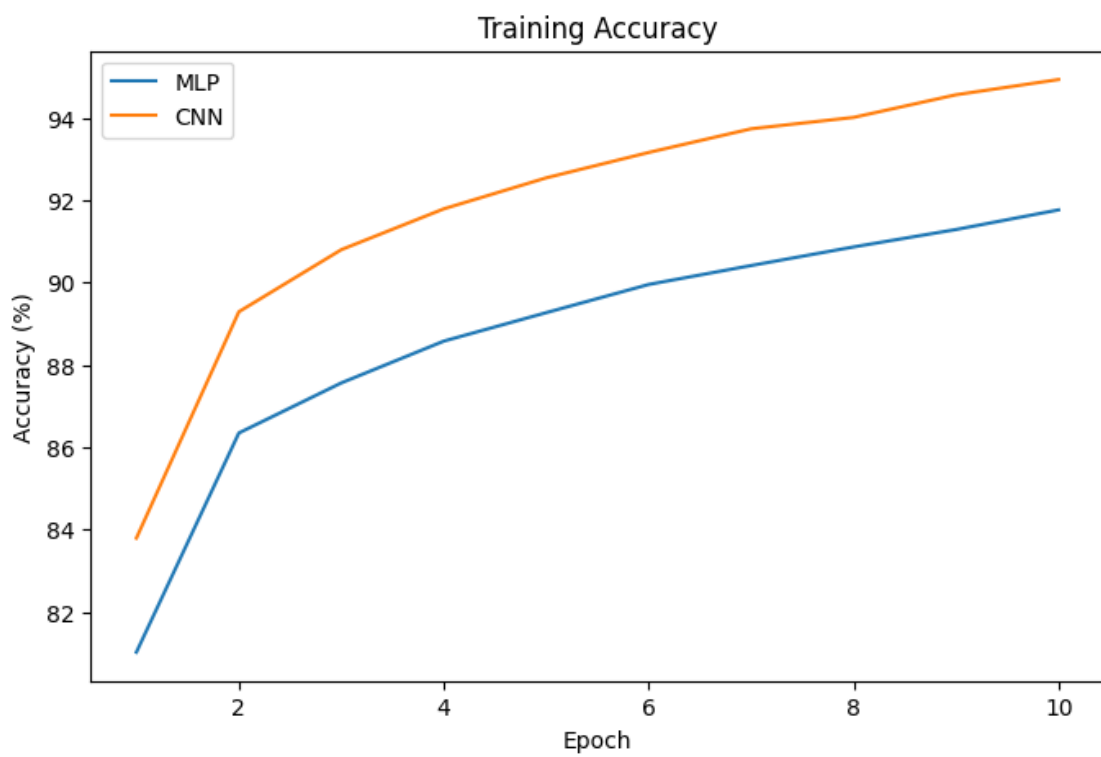
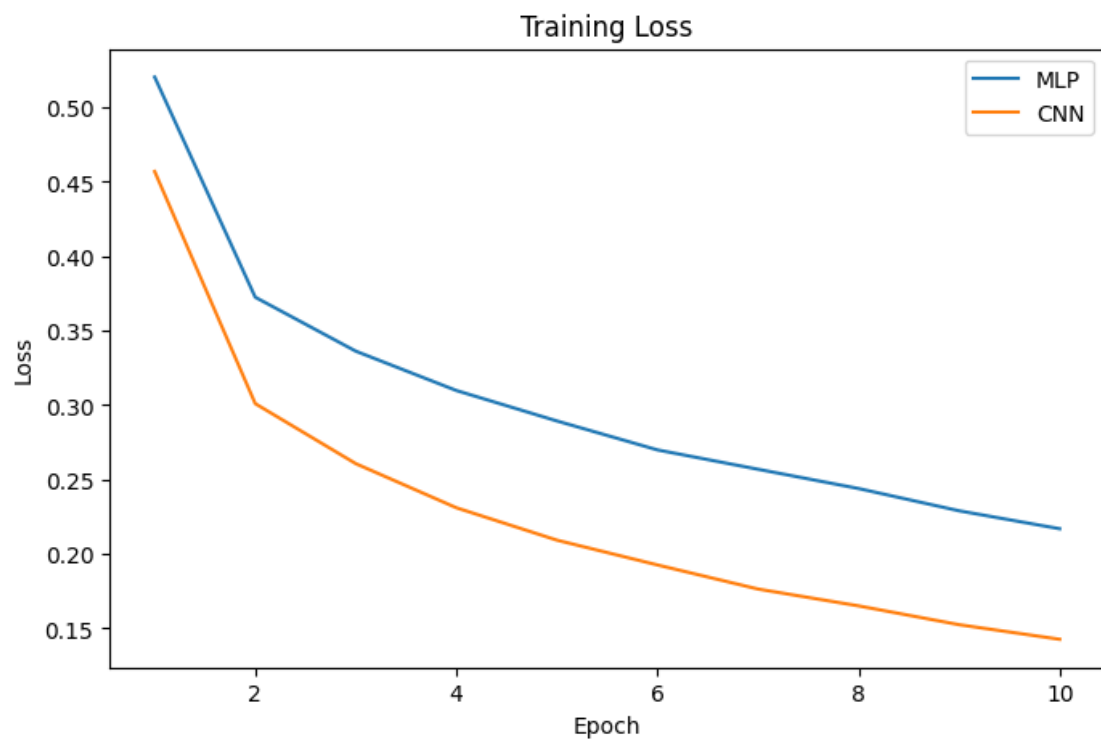
8 Plot

```
[17]: # Plot training progress
plt.figure(figsize=(8, 5))
plt.legend(['MLP', 'CNN'])
plt.title('Training Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.plot(
    range(1, num_epochs + 1),
    train_losses_mlp,
    label='MLP'
)
plt.plot(
    range(1, num_epochs + 1),
    train_losses_cnn,
    label='CNN'
)
plt.legend()
plt.show()

plt.figure(figsize=(8, 5))
plt.legend(['MLP', 'CNN'])
plt.title('Training Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy (%)')

plt.plot(
    range(1, num_epochs + 1),
    train_accs_mlp,
    label='MLP'
)
plt.plot(
    range(1, num_epochs + 1),
    train_accs_cnn,
    label='CNN'
)
plt.legend()
```

```
plt.show()
```



9 Visualize Predictions

```
[18]: # Visualize some predictions
def visualize_predictions(model, testloader, classes, num_images=5):
    model.eval()

    # Get a batch of test images
    dataiter = iter(testloader)
    images, labels = next(dataiter)
    images, labels = images[:num_images].to(device), labels[:num_images].
    ↪to(device)

    # Make predictions
    with torch.no_grad():
        outputs = model(images)
        _, predicted = torch.max(outputs, 1)

    # Move images back to CPU for visualization
    images = images.cpu()

    # Plot the images with their true and predicted labels
    fig = plt.figure(figsize=(15, 3))
    for i in range(num_images):
        ax = fig.add_subplot(1, num_images, i + 1)
        ax.imshow(images[i].squeeze(), cmap='gray')
        color = 'green' if predicted[i] == labels[i] else 'red'
        ax.set_title(f'True: {classes[labels[i]]}\nPred: ↪
    ↪{classes[predicted[i]]}', color=color)

    plt.tight_layout()
    plt.show()

# Visualize some predictions
visualize_predictions(model_mlp, testloader, classes)
```

